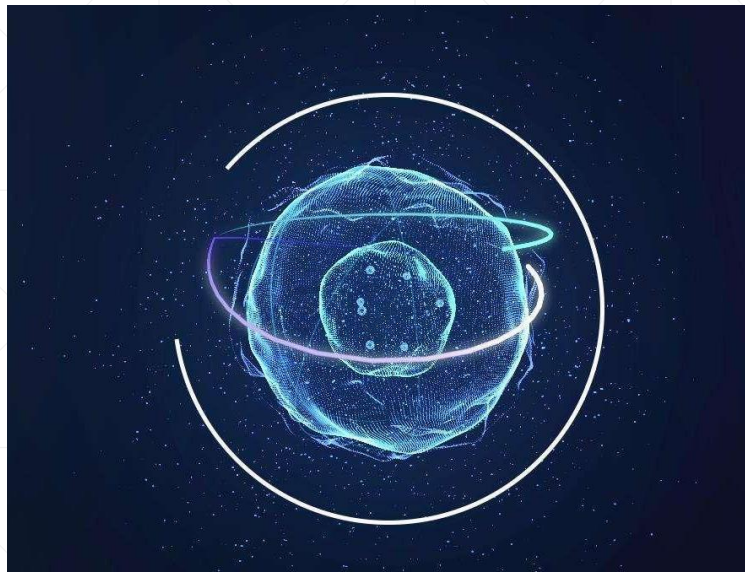




了解Stream API的技术特性

北京理工大学计算机学院
金旭亮

理解“Stream”



Stream API中的“Stream（流）”，可以看成是一组数据的集合，我们可以对它施加一系列的数据操作，这些操作可以是顺序进行的，也可以是并行执行的。



Stream看上去是集合，但实际上不是集合，为什么呢？

Stream不是集合-1



普通的集合，关注的是数据的存储方式，而Stream，关注的是施加于这些集合元素的处理任务（称为“**Operation（操作）**”）。

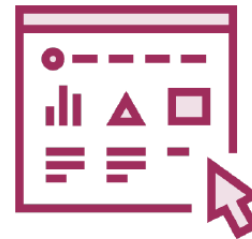


集合讲的是数据，流讲的是计算。



Stream通常从普通的集合中“抽取”数据，也可以从其它数据源（比如文件和数据库）中提取数据。

Stream不是集合-2



Stream只“抽取”数据，它从不会修改底层的数据源。简言之：
Stream Operation是只读操作！



只要底层数据源允许，流中包容的元素数目是不受限制的。



Stream API采用一种“**即抽取、即使用、即丢弃**”的方式处理数据，不需要“**把所有数据都加载到内存中（而普通集合就是这样的）**”才能工作，所以，能处理很大的数据集。

此Stream, 非彼Stream



Stream API中的“Stream”，与Java IO中的“Stream”含义是不一样的，Stream API中的Stream，可以看成一种“特殊的集合”，我们可以向这个集合施加一系列的“操作（Operation）”，而Java IO中的Stream，实际上是一个“数据流”，它是由一连串有序的数据（字节或字符）所构成的一个“数据序列”，是被加工的“原材料”。

流对象只能用一次!

```
//试图重用一個Stream, 引發異常
private static void StreamCannotBeReuse() {
    List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
    Stream<Integer> stream = numbers.stream();
    System.out.println("numbers集合中有元素" + stream.count() + "個");
    //以下這句將引發java.lang.IllegalStateException
    System.out.println("平均值為: " +
        stream.mapToInt(num -> num.intValue()).average(); 
);
}
```

↓ 引發

Exception in thread "main" java.lang.IllegalStateException:
stream has already been operated upon or closed

示例代碼: ReUseStream.java

处理“重用流”异常

```
//只有被重建之后, 才能再次执行操作
private static void StreamMustBeRecreated() {
    List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
    Stream<Integer> stream = numbers.stream();
    System.out.println("numbers集合中有元素" + stream.count() + "个");
    //重建一个流对象
    stream = numbers.stream();
    System.out.println("平均值为: "
        + stream.mapToInt(Integer::intValue)
        .average().getAsDouble());
}
```



numbers 集合中有元素5个
平均值为: 3.0

推荐的“重用流”的编程方式

当需要反复执行特定流的不同操作系列时，可以构建一个Supplier对象，通过它获取新的流实例。

```
//通过构建一个Supplier对象，重复创建特定的流
private static void StreamRecreatedViaSupplier() {
    List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
    Supplier<IntStream> streamSupplier =
        () -> numbers.stream().mapToInt(Integer::intValue);
    //使用Supplier接口所定义的get()方法，获取流对象的新实例
    System.out.println("numbers集合中有元素"
        + streamSupplier.get().count() + "个");
    System.out.println("平均值为: "
        + streamSupplier.get().average());
}
```



java.util.function包中提供了许多函数式的接口，Supplier就是其中的一个，在后面的课程中你还会遇到其他的函数式接口。

组合Stream API完成复杂的处理工作-1



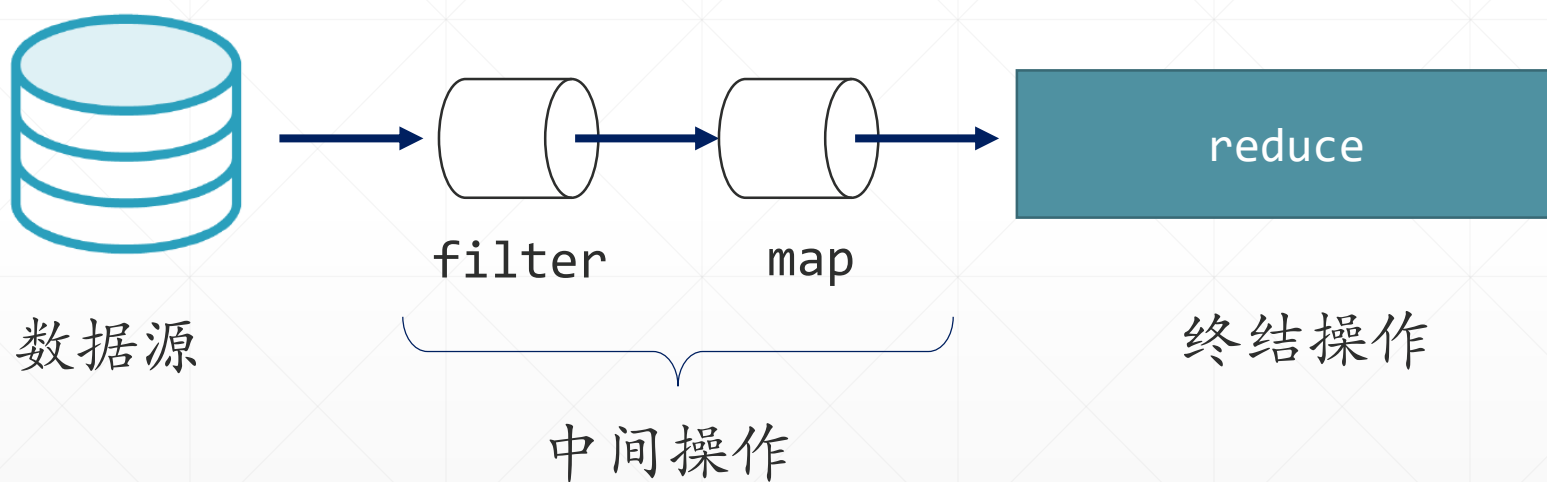
我们来看一个例子，求集合中所有奇数的平方和：

```
//以分步的方式展示Stream API如何完成复杂的数据处理任务
private static void seperateForm() {
    // 生成一个包容[1,5]区间所有整数的集合
    List<Integer> numbersList = Arrays.asList(1, 2, 3, 4, 5);
    // 将集合转变为流
    Stream<Integer> numbersStream = numbersList.stream();
    // 过滤提取出奇数
    Stream<Integer> oddNumbersStream= numbersStream.filter(n -> n % 2 == 1);
    // 求奇数的平方值 (映射/转换)
    Stream<Integer> squaredNumbersStream = oddNumbersStream.map(n -> n * n);
    // 累加求和 (归约)
    int sum = squaredNumbersStream.reduce( identity: 0, (n1, n2) -> n1 + n2);
    // 使用方法引用特性, 可以改写为:
    //int sum = squaredNumbersStream.reduce(0, Integer::sum);
    // 输出处理结果
    System.out.println(sum);
}
```

示例：SumOfOddNumbers.java

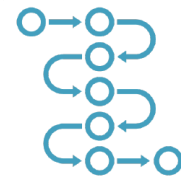
理解“流处理管线”

上述代码实际上构建了一个数据处理管线，如下图所示：



这条数据处理管线，可以使用一种“级联调用”的代码编写方式，更清晰明了地表现出来.....

使用“级联”调用方式简写代码



//以级联的方式编程

```
private static void compoundForm(){
```

```
    // 生成一个包容5个整数的集合
```

```
    List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
```

```
    // 计算集合中所有奇数平方值的总和
```

```
    int sum = numbers.stream()
```

```
        .peek(num-> System.out.println("\nfilter之前, 处理"+num))
```

```
        .filter(n -> n %2 ==1)
```

```
        .peek(num-> System.out.println("filter之后,map之前, 过滤出"+num))
```

```
        .map(n -> n * n)
```

```
        .peek(num-> System.out.println("map之后, reduce之前, 得到平方值: "+num))
```

```
        .reduce( identity: 0, Integer::sum);
```

```
    System.out.println("\n数据处理结束, 其结果为: "+sum);
```

```
}
```

级联调用的Stream API
数据处理函数

为了清晰地展示出数据的处理流程，加上了peek方法，表明当前正在处理哪个元素。

数据集[1, 2, 3, 4, 5]，要计算所有奇数平方值的总和

```
Run: SumOfOddInteger x
"C:\Program Files\Java\jdk-16.0.2\bin\java.exe"

filter之前, 处理1
filter之后, map之前, 过滤出1
map之后, reduce之前, 得到平方值: 1

filter之前, 处理2
filter之后, map之前, 过滤出3
map之后, reduce之前, 得到平方值: 9

filter之前, 处理4
filter之后, map之前, 过滤出5
map之后, reduce之前, 得到平方值: 25

数据处理结束, 其结果为: 35

Process finished with exit code 0
```

1是奇数，过滤出来之后，立即传给map进行转换，得到结果，缓存起来.....

2是偶数，不符合要求，跳过

3是奇数，符合条件，传给map处理

4是偶数，不符合要求，跳过

5是奇数，符合条件，传给map处理

数据预处理结束，进行归约，最后得到结果

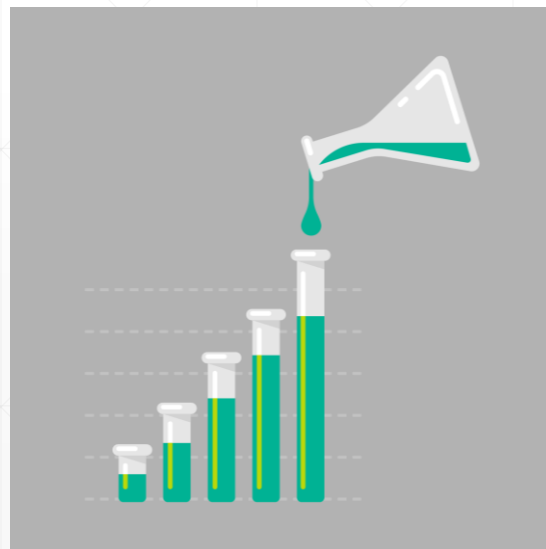
凡是偶数，都会被“剔除”，不参与后继的处理流程，这就是“短路”的实例。

构成流处理管道的两种操作类型

流处理管道由“**中间操作**”和“**终结（终端、终止）操作**”所构成。数据流入“中间操作”，中间操作可以对这些数据进行特定的加工和处理，再将其转发给下一个操作，就象工厂中的流水线一样。

终止操作完成以下任务：

1. 启动执行整个流操作系列
2. 向调用者返回最终结果
3. 操作结束后，销毁流对象



整理学过的流的相关知识-1



流不是集合，它所包容的元素来自于集合，是按需计算的，并且是只读访问。



流采用内部迭代方式遍历元素，并且只能遍历一次，遍历完之后，我们就说这个流已经被消费掉了。



Stream API采用声明式编程风格，使用级联的方式构建数据处理管线，管线中间的各个数据处理操作要等到最后再“批处理”执行，这个特性称为“**惰性 (lazy) 求值**”，并且支持并行执行。



Stream API支持“**短路**”特性，当流中某个元素不符合特定要求时，它不会流入到后继的处理环节中，需要时也可以提前结束整个数据处理流程。

整理学过的流的相关知识-2



使用Stream，开发者可以很方便地让代码并行执行而无需显式编写多线程代码。



基于Stream的代码，具有“**函数式编程 (functional programming)**”风格。



Stream只能使用一次，如果希望再次执行这些操作，必须创建一个新的Stream。



Stream API中定义的数据处理函数，通常可以级联调用，构成一个数据处理链条。